Calibration Language Changes How Quantized Models Fail: A Case Study of a Small Bangla Math LLM

Anonymous ACL submission

Abstract

Quantization lets large language models (LLMs) run on low-cost hardware, often the only option for speakers of languages such as Bangla. Yet quantization is calibrated almost only on English, and its effect on other languages is poorly understood, although Bangla needs 2.9× more tokens per word, giving quantization error more steps to accumulate. We ask how quantization, and the language of its calibration data, affect a small Bangla math model. We quantize Gemma-2 2B, fine-tuned on Bangla math, with round-to-nearest (RTN), NF4 and GPTQ at 8 to 3 bits, calibrate GPTQ on five sets including the same problems in Bangla and in English, and test 200 held-out problems with corrected paired tests. 8-bit RTN is lossless, 3-bit RTN solves nothing, and GPTQ beats RTN at 4 and 3 bits. Calibration language does not significantly change accuracy, but English calibration raises the share of repetitive, unfinished outputs by about 10 points ($p < 0.001$), even on identical problems, while changing only the domain has no effect. Conceptually, quantization damage in a low-resource language appears as repetition rather than language drift, and accuracy alone misses it. Practically, deployments should calibrate in the target language and monitor repetition.

1 Introduction

Bangla is spoken by more than 230 million people, most of whom have limited access to the GPUs needed to run modern LLMs. Quantization stores weights in 8, 4 or fewer bits instead of 16, which cuts memory and cost (Dettmers et al., 2022; Frantar et al., 2023).

Quantization adds rounding error to every weight. The best methods, such as GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024), reduce this error using a small *calibration set* of example inputs, which in practice is usually English web text. For a model used in Bangla, this default is questionable for two reasons. First, the model's internal activations on Bangla text may differ from those on English. Second, Bangla is split into far more tokens per word than English (Petrov et al., 2023; Ahia et al., 2023), so a solution takes more generation steps, and a small error has more chances to derail it. Step-by-step mathematics is especially exposed, because one wrong step makes the final answer wrong.

Prior work shows that quantization hurts non-Latin-script languages more than English (Marchisio et al., 2024) and that calibration data matter for compressed models (Williams and Aletras, 2024; Zeng et al., 2024). We study the question in a concrete setting: one small model, fine-tuned for one task in one low-resource language. We make three contributions:

1. We compare seven quantization configurations from 8 to 3 bits and show that the quantization method matters far more than the bit width alone: GPTQ keeps nearly all accuracy at 4 bits, while plain rounding collapses at 3 bits.

2. We separate the effect of calibration *language* from *domain*, including a parallel design that calibrates on the same problems in Bangla and in English. Calibration language has no significant effect on accuracy but a large, significant effect on how often outputs fall into repetition loops.

3. We show that accuracy hides much of what quantization changes: models fail by repeating themselves rather than by switching to English, and even an 8-bit model with unchanged accuracy solves a noticeably different set of problems, well beyond what numerical noise explains.

2 Related Work

Post-training quantization. LLM.int8 (Dettmers et al., 2022) stores weights in 8 bits and keeps outlier features in 16 bits. NF4 (Dettmers et al., 2023) places 16 levels according to a normal distribution and is the base format of QLoRA fine-tuning. GPTQ (Frantar et al., 2023) quantizes weights column by column and uses calibration inputs to correct the error of each step. AWQ (Lin et al., 2024) and SmoothQuant (Xiao et al., 2023) also rely on calibration activations.

Calibration data and languages. Williams and Aletras (2024) show that the choice of calibration data changes the downstream performance of quantized and pruned models. Zeng et al. (2024) show that English-only calibration disadvantages other languages when compressing multilingual models. Marchisio et al. (2024) find that quantization harms non-Latin-script languages most and that automatic metrics underestimate the damage. We add a parallel design that holds the calibration problems fixed and changes only their language, and we measure how failures change, not only accuracy.

Bangla mathematical reasoning. English math benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) have Bangla counterparts only recently; BEnQA (Shafayat et al., 2024) shows a large gap between Bangla and English reasoning even for strong models. We use a public Bangla math corpus with step-by-step solutions (hamim-87, 2024), which is a Bangla translation of NuminaMath-CoT (Li et al., 2024).

3 Experimental Setup

Model. We fine-tune Gemma-2 2B-it (Gemma Team, 2024) with QLoRA (Dettmers et al., 2023; Hu et al., 2022) on 3,000 problem–solution pairs from the Bangla dataset. LoRA (rank 16, $\alpha = 16$) is applied to all attention and MLP projections; we train for 400 steps with effective batch size 8 and learning rate 2×10^{-4}. The adapter is merged into the 16-bit model, which serves as the reference and as the input to every quantization method. Of the model's 2.61B parameters, 2.02B (77%) are in the linear layers of the transformer blocks; only these are quantized, as is standard.

Quantization methods. **RTN** rounds each weight to the nearest of 2^b levels, with a scale and zero point per output row (8-bit) or per group of 128 weights (4 and 3-bit). **NF4** uses the 16 levels of Dettmers et al. (2023) with one scale per block of 64 weights. **GPTQ** (Frantar et al., 2023) uses the same groups as RTN but quantizes columns in order and adjusts the remaining weights to compensate for each rounding error, using statistics from the calibration set. Comparing GPTQ with RTN at equal size therefore isolates the value of calibration.

We use *simulated* quantization: weights are rounded and stored back in 16-bit, so the model computes with exactly the values a low-bit kernel would use. This is standard for measuring accuracy and lets every method run on the same machine (Apple M1 Pro (MPS)). Sizes are computed from the bit widths; we do not report speed, which depends on hardware kernels rather than on the method.

Calibration set	Language	Content
Bangla math	Bangla	training problems
English GSM8K	English	grade-school math
English Wikipedia	English	WikiText-2 text
Bangla parallel	Bangla	matched problems
English parallel	English	same, original English

Table 1: GPTQ calibration sets. English GSM8K vs. English Wikipedia changes only the domain; the parallel pair changes only the language.

Calibration sets. Table 1 lists the five calibration sets, each with 128 samples of up to 512 tokens. The first three follow common practice; English Wikipedia is WikiText-2 (Merity et al., 2017). The last two form a *parallel* pair: we matched Bangla training problems to their English originals in NuminaMath-CoT by their numbers and final answers (1,370 unique matches), so the two sets contain exactly the same problems and differ only in language. Each 4-bit configuration is quantized with 3 random samples (seeds); 3-bit GPTQ uses one seed.

Evaluation. The test set has 200 problems drawn from rows never used in training. We keep problems with an automatically checkable answer and a reference solution of at most 384 tokens, about a third of the data, so that a complete solution fits in the generation budget. The answers are numbers (120 (60%)), multiple-choice letters (43 (22%)) or short expressions (37 (18%)). Models decode greedily for up to 768 tokens. A prob-

lem counts as solved if the last \boxed{} answer matches the reference after normalization (Bangla digits, LaTeX formatting, units, numeric tolerance, and choice letters matched to option values).

For failed problems we record the failure type: a *wrong* answer, no answer within the token *limit*, or *stopped* without an answer. We also mark an output as *repetitive* if any sequence of eight words occurs three or more times, and as *drifting* if fewer than half of its letters are Bangla script.

Statistics. All models answer the same problems, so we compare them with a paired bootstrap over problems (Koehn, 2004) (10,000 resamples), averaging over seeds first. We correct for multiple comparisons with the Holm method within each research question: all methods against 16-bit; GPTQ against RTN; and the calibration comparisons, separately for accuracy and repetition. To separate quantization effects from numerical noise, we run the 16-bit model a second time with a different batch size.

4 Results

4.1 The method matters more than the bit width

Table 2 summarizes accuracy and size. The 16-bit model solves 17.5% of the problems.

8 bits cost nothing. RTN 8-bit shrinks the model from 4.87 to 2.99 GB (39% smaller) with identical accuracy.

At 4 bits, calibration-based correction helps. Plain RTN drops to 10.5%, while GPTQ keeps $16.7 \pm 1.2\%$ at the same size, a difference of $+6.2$ pp (95% CI [+1.7, +10.7]) (Holm $p = 0.007$). NF4 lies in between.

At 3 bits, RTN collapses. It solves no problem at all, while 3-bit GPTQ still solves 14.5% (difference $+14.5$ pp (95% CI [+10.0, +19.5]), $p < 0.001$).

Against 16-bit, only the 3-bit RTN drop survives the Holm correction; 4-bit losses of 0.2 to 7 points are too small to confirm with 200 problems.

4.2 Calibration language changes failures, not accuracy

Figure 1 compares the five calibration sets for 4-bit GPTQ.

Accuracy. Bangla calibration gives the highest accuracy in both designs: $16.7 \pm 1.2\%$ for Bangla math versus $13.0 \pm 0.9\%$ for English Wikipedia, and $17.3 \pm 0.8\%$ versus $15.7 \pm 3.3\%$ for the parallel pair. None of these differences is significant after correction (Bangla math vs. English Wikipedia: $+3.7$ pp (95% CI [+0.7, +6.8]), Holm $p = 0.078$; parallel pair: $+1.7$ pp (95% CI [-2.0, +5.3]), $p = 0.782$). Seed-to-seed differences are as large as differences between sets (English parallel: 12.5–19.0%).

Repetition. The language of the calibration set has a clear effect on how the model fails. With Bangla calibration, 43–44% of outputs contain a repetition loop, the same as the 16-bit model (44.5%). With any English calibration set, 52–54% do. On identical problems, switching the calibration language from English to Bangla reduces repetitive outputs by -10.2 pp (95% CI [-15.2, -5.2]) (Holm $p < 0.001$). Changing only the domain, from English Wikipedia to English math, has no effect ($+1.5$ pp (95% CI [-3.7, +6.7]), $p = 0.575$). The cause is the calibration language, not the topic.

4.3 Quantized models repeat; they do not drift

Table 3 shows how models fail. Even the 16-bit model fails in two main ways: a wrong answer (39.5%) or no answer within the token limit (39.5%). The second kind is almost always a loop: 82–99% of outputs that hit the limit are repetitive, in every configuration.

Quantization error shifts failures toward these loops. Unfinished outputs rise from 39.5% (16-bit) to 52.5% with RTN 4-bit and 98.5% with RTN 3-bit, and outputs grow longer (from 499 to 565 and 761 tokens on average). A typical RTN 3-bit output ends by repeating, in Bangla, "…to consider how many possible cases of the divisor there are, based on the relation between dividend and remainder, we…" until the budget runs out.

Language drift, by contrast, is negligible: at most 0.5% of outputs are mostly non-Bangla in any configuration, including the English-calibrated ones. English calibration damages the coherence of long Bangla generations, not the choice of language.

4.4 Equal accuracy hides different answers

The last column of Table 2 shows that quantized models solve different problems, not just fewer. RTN 8-bit has the same accuracy as 16-bit, yet it fails 23% of the problems the 16-bit model solved and solves 5% of those it failed; for 4-bit GPTQ

Method	Bits/weight	Size (GB)	Accuracy (%)	Δ vs. 16-bit (pp) [95% CI]	Lost / gained (%)
16-bit (reference)	16.00	4.87	17.5	–	–
RTN 8-bit	8.01	2.99	17.5	+0.0 [-4.0, +4.0]	23 / 5
NF4 4-bit	4.50	2.16	15.0	−2.5 [-8.0, +3.0]	51 / 8
RTN 4-bit	4.16	2.08	10.5	−7.0 [-12.5, -1.5]	66 / 5
GPTQ 4-bit, Bangla math	4.16	2.08	16.7 ± 1.2	−0.8 [-5.2, +3.3]	45 / 8
GPTQ 4-bit, English GSM8K	4.16	2.08	13.8 ± 2.3	−3.7 [-8.5, +1.0]	59 / 8
GPTQ 4-bit, English wiki	4.16	2.08	13.0 ± 0.9	−4.5 [-9.2, 0.0]	58 / 7
GPTQ 4-bit, Bangla parallel	4.16	2.08	17.3 ± 0.8	−0.2 [-4.3, +4.0]	42 / 9
GPTQ 4-bit, English parallel	4.16	2.08	15.7 ± 3.3	−1.8 [-6.7, +3.0]	57 / 10
RTN 3-bit	3.15	1.84	0.0	−17.5* [-23.0, -12.5]	100 / 0
GPTQ 3-bit, Bangla math	3.15	1.84	14.5	−3.0 [-8.5, +2.5]	54 / 8

Table 2: Accuracy and size on 200 held-out Bangla math problems. 4-bit GPTQ rows are mean ± std over 3 calibration seeds. Δ: paired-bootstrap difference; * = significant after Holm correction over the ten comparisons with 16-bit. Lost / gained: share of problems solved by 16-bit that the quantized model fails, and share of 16-bit failures it solves.

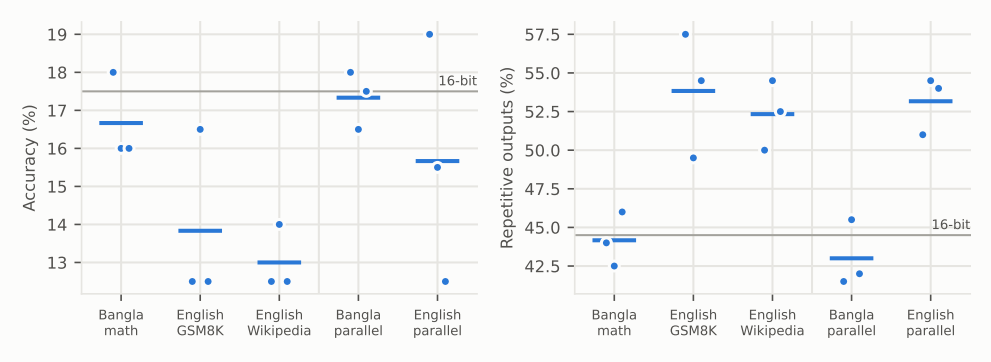


Figure 1: 4-bit GPTQ under each calibration set. Left: accuracy. Right: share of outputs that fall into repetition loops. Dots are calibration seeds, bars are means, grey lines show the 16-bit model. The two rightmost sets contain the same problems in Bangla and English.

Method	Wrong	Limit	Stopped	Repet.	Drift	Tokens
16-bit (reference)	39.5	39.5	3.5	44.5	0.5	499
RTN 8-bit	38.5	41.5	2.5	42.0	0.5	499
NF4 4-bit	44.5	37.0	3.5	44.5	0.5	505
RTN 4-bit	33.5	52.5	3.5	58.5	0.5	565
GPTQ 4-bit, Bangla math	40.8	39.0	3.5	44.2	0.0	510
GPTQ 4-bit, English GSM8K	35.7	46.3	4.2	53.8	0.0	537
GPTQ 4-bit, English wiki	36.5	46.2	4.3	52.3	0.0	536
GPTQ 4-bit, Bangla parallel	40.0	39.2	3.5	43.0	0.0	520
GPTQ 4-bit, English parallel	33.3	47.2	3.8	53.2	0.0	538
RTN 3-bit	0.5	98.5	1.0	97.5	0.0	761
GPTQ 3-bit, Bangla math	34.0	46.0	5.5	53.5	0.0	536

Table 3: Failure types, % of problems. *Wrong*: incorrect boxed answer. *Limit*: no answer within 768 tokens. *Stopped*: ended without an answer. *Repet.*: output contains a repetition loop. *Drift*: mostly non-Bangla output. *Tokens*: mean output length.

with Bangla calibration the figures are 45% and 8%. This churn is not numerical noise: rerunning the 16-bit model with a different batch size reproduces 87% of outputs exactly and changes only 3% of solved problems.

4.5 Why long outputs matter for Bangla

Gemma's tokenizer uses 3.49 tokens per word on Bangla solutions and 1.18 on English text (2.9×); the median reference solution is 511 tokens long. Each generation step is a chance for quantization error to start a loop, consistent with damage appearing mainly as unfinished, repetitive output.

5 Conclusion

For a small Bangla math model, the quantization method matters more than the bit width, and the calibration language changes how the model fails rather than how often it succeeds: English calibration makes outputs fall into repetition loops about ten points more often, even on identical problems. For low-resource deployments, we recommend calibrating in the target language, comparing several calibration seeds, and monitoring repetition and unfinished answers, which accuracy and language-identification checks both miss.

Limitations

We study one model (Gemma-2 2B) fine-tuned on one machine-translated dataset, so results may not transfer to larger models, other languages or other tasks. The test set contains 200 problems with the shortest third of reference solutions; with this sample, accuracy differences of a few points cannot be confirmed, and 3-bit GPTQ was run with a single seed. The repetition measure was defined after we observed that most unfinished outputs were loops, so the repetition comparisons are a secondary analysis, although they are corrected for multiple testing. One of the 24 GPTQ runs was interrupted by memory pressure and resumed after re-quantizing with the same seed and calibration data; we did not verify that the two quantizations are bit-identical. We use simulated quantization, which reproduces accuracy but not speed, and does not model LLM.int8's outlier handling or NF4's double quantization. Answer matching is automatic and ignores the quality of the reasoning and the Bangla prose. We do not study activation quantization, AWQ, or 2-bit weights.

Ethical Considerations

The model is a study object, not a tutoring system: even before quantization it solves only 17.5% of the test problems and can produce fluent but wrong solutions. All data are public and contain no personal information.

Acknowledgments

We used an AI assistant (Claude, Anthropic) for help with code and for editing the text. The authors checked all code, results and text and take full responsibility for them.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. 2023. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*.

Gemma Team. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

hamim-87. 2024. Ashrafur bangla math. Hugging Face dataset, https://huggingface.co/datasets/hamim-87/Ashrafur_bangla_math.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. NuminaMath. https://huggingface.co/datasets/AI-MO/NuminaMath-CoT.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. 2024. How does quantization affect multilingual LLMs? In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Sheikh Shafayat, H. M. Quamran Hasan, Minhajur Rahman Chowdhury Mahim, Rifki Afina Putri, James Thorne, and Alice Oh. 2024. BEnQA: A question answering benchmark for Bengali and English. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Miles Williams and Nikolaos Aletras. 2024. On the impact of calibration data in post-training quantization and pruning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*.

Hongchuan Zeng, Hongshen Xu, Lu Chen, and Kai Yu. 2024. Multilingual brain surgeon: Large language models can be compressed leaving no language behind. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*.